

MODUL 14

Nama: Ryan Akeyla Novianto Widodo

Kelas: 12 IF 07

NIM : 103112400081

DASAR TEORI

Dasar Teori C++

Diambil dari internet dan dirangkum ulang oleh saya dan dibantu dengan AI.

1. Inti Program

- Syntax: Mirip bahasa Inggris, tapi huruf besar dan kecil beda arti. Setiap perintah biasanya diakhiri dengan titik koma (;).

- Komentar: Catatan di kode yang tidak dijalankan.

- Satu baris: `// Ini komentar`

- Banyak baris: `/* Ini komentar yang panjang */`

- Header: File yang berisi daftar perintah dan fungsi yang bisa dipakai. Contoh:

cpp

```
#include <iostream> // Untuk perintah input/output (cout, cin)
```

- `main()` : Fungsi utama, tempat program mulai berjalan.

cpp

```
int main() {
```

```
    // Kode program di sini
```

```
    return 0; // Menandakan program selesai dengan benar
```

```
}
```

- Namespace: Seperti "area" untuk nama-nama perintah. `std` adalah namespace standar C++.

cpp

```
using namespace std; // Supaya tidak perlu tulis std::cout, std::cin, dll.
```

- Input/Output:

- cout : Menampilkan sesuatu di layar.

- cin : Membaca input dari keyboard.

cpp

```
cout << "Halo!" << endl; // Menampilkan "Halo!"
```

```
int umur;
```

```
cin >> umur; // Membaca angka dari keyboard dan disimpan di variabel "umur"
```

2. Jenis Data

- Integer (Bilangan Bulat):

- int : Bilangan bulat (contoh: -10, 0, 25).

- short : Bilangan bulat pendek.

- long : Bilangan bulat panjang.

- Floating-Point (Bilangan Desimal):

- float : Bilangan desimal (contoh: 3.14).

- double : Bilangan desimal dengan ketelitian lebih tinggi.

- Character (Karakter):

- char : Satu huruf, angka, atau simbol (contoh: 'a', 'Z', '5').

- Boolean (Logika):

- bool : Nilai benar (true) atau salah (false).

- void : "Kosong". Dipakai untuk fungsi yang tidak menghasilkan nilai.

3. Variabel

- Deklarasi: Memberi tahu program nama dan jenis data variabel.

cpp

```
int umur; // Variabel bernama "umur" untuk menyimpan bilangan bulat
```

```
double harga; // Variabel bernama "harga" untuk menyimpan bilangan desimal
```

- Inisialisasi: Memberi nilai awal ke variabel.

cpp

```
int umur = 20; // "umur" diisi dengan nilai 20
```

```
double harga = 99.50; // "harga" diisi dengan nilai 99.50
```

4. Operator

- Aritmatika: + (tambah), - (kurang), * (kali), / (bagi), % (sisa bagi).
- Penugasan: = (mengisi nilai ke variabel).
- Perbandingan: == (sama dengan), != (tidak sama dengan), > (lebih besar), < (lebih kecil), >= (lebih besar atau sama dengan), <= (lebih kecil atau sama dengan).
- Logika: && (DAN), || (ATAU), ! (TIDAK).
- Increment/Decrement: ++ (tambah 1), -- (kurang 1).

5. Struktur Kontrol

- if : Melakukan sesuatu jika kondisi benar.

cpp

```
if (umur >= 17) {  
    cout << "Anda sudah dewasa." << endl;  
} else {  
    cout << "Anda belum dewasa." << endl;  
}
```

- switch : Memilih salah satu dari beberapa pilihan.

cpp

```
int hari = 3;  
switch (hari) {  
    case 1:  
        cout << "Senin" << endl;  
        break;  
    case 2:
```

```

        cout << "Selasa" << endl;

        break;

    case 3:

        cout << "Rabu" << endl;

        break;

    default:

        cout << "Hari lain" << endl;

}

```

- for : Melakukan sesuatu berulang kali sebanyak yang ditentukan.

cpp

```

for (int i = 0; i < 5; i++) {

    cout << i << " "; // Menampilkan angka 0 sampai 4

}

cout << endl;

```

- while : Melakukan sesuatu berulang kali selama kondisi benar.

cpp

```

int angka = 1;

while (angka <= 3) {

    cout << angka << " ";

    angka++;

}

cout << endl;

```

- do-while : Sama seperti while , tapi dilakukan minimal sekali.

cpp

```

int pilihan;

do {

    cout << "Pilih 1, 2, atau 3: ";

```

```
cin >> pilihan;  
} while (pilihan < 1 || pilihan > 3);
```

6. Fungsi

- Definisi: Sekumpulan kode yang melakukan tugas tertentu.

cpp

```
int tambah(int a, int b) {  
    return a + b;  
}
```

- Pemanggilan: Menggunakan fungsi.

cpp

```
int hasil = tambah(5, 2); // Memanggil fungsi "tambah" dengan angka 5 dan 2  
cout << "Hasil: " << hasil << endl; // Menampilkan "Hasil: 7"
```

7. Pointer

- Definisi: Variabel yang menyimpan alamat memori variabel lain. (Agak rumit, tapi penting).

cpp

```
int umur = 25;  
  
int *p_umur = &umur; // p_umur menyimpan alamat memori dari variabel umur
```

DASAR TEORI GRAPH

Pengertian Graph

Graph (graf) adalah struktur data yang digunakan untuk merepresentasikan hubungan antar objek.

Graph terdiri dari kumpulan simpul (vertex/node) dan kumpulan sisi (edge) yang menghubungkan antar simpul.

Graph merupakan struktur data yang digunakan untuk merepresentasikan hubungan antar objek dalam bentuk simpul (vertex) dan sisi (edge). Dalam bahasa C++, graph dapat diimplementasikan dengan beberapa cara seperti *adjacency matrix* dan *adjacency list*, tergantung kebutuhan efisiensi memori dan kemudahan pengolahan. Graph sangat penting karena mampu memodelkan berbagai permasalahan nyata seperti jaringan komputer, peta, dan sistem relasi data. Dengan memahami konsep dan operasi dasar graph, programmer dapat menyelesaikan masalah yang melibatkan keterkaitan antar data secara lebih efektif dan terstruktur.

Guided

```
#include <iostream>
#include <queue>
using namespace std;

/* =====
   TYPE & STRUCT GRAPH
   ===== */

typedef char infoGraph;

struct ElmNode;
struct ElmEdge;

typedef ElmNode* adrNode;
typedef ElmEdge* adrEdge;

struct ElmEdge {
    adrNode node;
    adrEdge next;
};

struct ElmNode {
    infoGraph info;
    int visited;
    adrEdge firstEdge;
    adrNode next;
};

struct Graph {
    adrNode first;
};

/* =====
   PRIMITIF GRAPH
   ===== */

void CreateGraph(Graph &G) {
    G.first = NULL;
}

adrNode AllocateNode(infoGraph X) {
    adrNode P = new ElmNode;
    P->info = X;
    P->visited = 0;
    P->firstEdge = NULL;
    P->next = NULL;
    return P;
}
```

```

adrEdge AllocateEdge(adrNode N) {
    adrEdge P = new ElmEdge;
    P->node = N;
    P->next = NULL;
    return P;
}

void InsertNode(Graph &G, infoGraph X) {
    adrNode P = AllocateNode(X);
    P->next = G.first;
    G.first = P;
}

adrNode FindNode(Graph G, infoGraph X) {
    adrNode P = G.first;
    while (P != NULL) {
        if (P->info == X)
            return P;
        P = P->next;
    }
    return NULL;
}

void ConnectNode(Graph &G, infoGraph A, infoGraph B) {
    adrNode N1 = FindNode(G, A);
    adrNode N2 = FindNode(G, B);

    if (N1 == NULL || N2 == NULL) {
        cout << "Node tidak ditemukan!\n";
        return;
    }

    // Edge A → B
    adrEdge E1 = AllocateEdge(N2);
    E1->next = N1->firstEdge;
    N1->firstEdge = E1;

    // Edge B → A (undirected)
    adrEdge E2 = AllocateEdge(N1);
    E2->next = N2->firstEdge;
    N2->firstEdge = E2;
}

void PrintInfoGraph(Graph G) {
    adrNode P = G.first;
    while (P != NULL) {
        cout << P->info << " -> ";
        adrEdge E = P->firstEdge;
        while (E != NULL) {

```

```

        cout << E->node->info << " ";
        E = E->next;
    }
    cout << endl;
    P = P->next;
}
}

/* =====
   TRAVERSAL
   ===== */

void ResetVisited(Graph &G) {
    adrNode P = G.first;
    while (P != NULL) {
        P->visited = 0;
        P = P->next;
    }
}

void PrintDFS(Graph &G, adrNode N) {
    if (N == NULL) return;

    N->visited = 1;
    cout << N->info << " ";

    adrEdge E = N->firstEdge;
    while (E != NULL) {
        if (E->node->visited == 0) {
            PrintDFS(G, E->node);
        }
        E = E->next;
    }
}

void PrintBFS(Graph &G, adrNode N) {
    if (N == NULL) return;

    queue<adrNode> Q;
    Q.push(N);

    while (!Q.empty()) {
        adrNode curr = Q.front();
        Q.pop();

        if (curr->visited == 0) {
            curr->visited = 1;
            cout << curr->info << " ";

            adrEdge E = curr->firstEdge;

```

```

        while (E != NULL) {
            if (E->node->visited == 0) {
                Q.push(E->node);
            }
            E = E->next;
        }
    }
}

/* =====
MAIN PROGRAM
===== */

int main() {
    Graph G;
    CreateGraph(G);

    // Tambah node
    InsertNode(G, 'A');
    InsertNode(G, 'B');
    InsertNode(G, 'C');
    InsertNode(G, 'D');
    InsertNode(G, 'E');

    // Hubungkan node (graph tidak berarah)
    ConnectNode(G, 'A', 'B');
    ConnectNode(G, 'A', 'C');
    ConnectNode(G, 'B', 'D');
    ConnectNode(G, 'C', 'E');

    cout << "=== Struktur Graph ===" << endl;
    PrintInfoGraph(G);

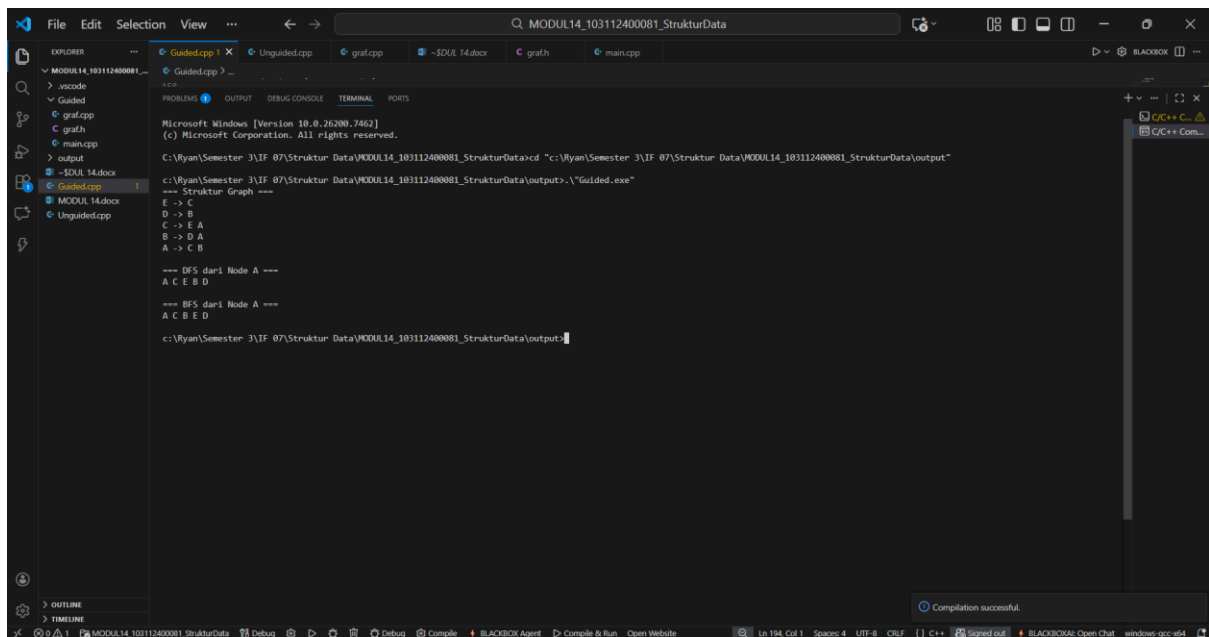
    cout << "\n=== DFS dari Node A ===" << endl;
    ResetVisited(G);
    PrintDFS(G, FindNode(G, 'A'));

    cout << "\n\n=== BFS dari Node A ===" << endl;
    ResetVisited(G);
    PrintBFS(G, FindNode(G, 'A'));

    cout << endl;
    return 0;
}

```

Hasil Coding:



The screenshot shows a Visual Studio IDE with a C++ project named 'MODUL14_103112400081_StrukturData'. The file explorer on the left shows the project structure, including 'main.cpp' and 'output'. The terminal window displays the output of the program, which includes the graph structure and the results of DFS and BFS traversals.

```
Microsoft Windows [Version 10.0.26200.7462]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Ryan\Semester 3\IF 07\Struktur Data\MODUL14_103112400081_StrukturData>cd "c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL14_103112400081_StrukturData\output"  
c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL14_103112400081_StrukturData\output>.\"Guided.exe"  
=== Struktur Graph ===  
E -> C  
D -> B  
C -> E A  
B -> D A  
A -> C B  
  
=== DFS dari Node A ===  
A C E B D  
  
=== BFS dari Node A ===  
A C B E D  
  
c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL14_103112400081_StrukturData\output>
```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita membangun dan mengelola sebuah struktur data graph (graf) tidak berarah menggunakan representasi adjacency list, serta melakukan penelusuran graph menggunakan algoritma DFS (Depth First Search) dan BFS (Breadth First Search).

Unguided

Coding:

```
#include <iostream>
using namespace std;

typedef char charInfo;

struct Vertex;
struct Edge;

typedef Vertex* ptrVertex;
typedef Edge* ptrEdge;

struct Edge {
    ptrVertex destVertex;
    ptrEdge nextEdge;
};

struct Vertex {
    charInfo id;
    int status;
    ptrEdge firstIncidentEdge;
    ptrVertex nextVertex;
};

struct Graph {
    ptrVertex firstVertex;
};

struct NodeQ {
    ptrVertex val;
    NodeQ* next;
};

struct QueueList {
    NodeQ* head;
    NodeQ* tail;
};

void createQ(QueueList &Q) {
    Q.head = NULL;
    Q.tail = NULL;
}

bool emptyQ(QueueList Q) {
    return (Q.head == NULL);
}

void enq(QueueList &Q, ptrVertex v) {
```

```

NodeQ* baru = new NodeQ;
baru->val = v;
baru->next = NULL;

if (emptyQ(Q)) {
    Q.head = baru;
    Q.tail = baru;
} else {
    Q.tail->next = baru;
    Q.tail = baru;
}
}

ptrVertex deq(QueueList &Q) {
    if (emptyQ(Q)) return NULL;

    NodeQ* del = Q.head;
    ptrVertex out = del->val;

    Q.head = Q.head->next;
    if (Q.head == NULL) Q.tail = NULL;

    delete del;
    return out;
}

void initGraph(Graph &G) {
    G.firstVertex = NULL;
}

ptrVertex alokasiVertex(charInfo data) {
    ptrVertex V = new Vertex;
    V->id = data;
    V->status = 0;
    V->firstIncidentEdge = NULL;
    V->nextVertex = NULL;
    return V;
}

ptrEdge alokasiEdge(ptrVertex tujuan) {
    ptrEdge E = new Edge;
    E->destVertex = tujuan;
    E->nextEdge = NULL;
    return E;
}

void addVertex(Graph &G, charInfo data) {
    ptrVertex V = alokasiVertex(data);

    if (G.firstVertex == NULL) {

```

```

        G.firstVertex = V;
    } else {
        ptrVertex P = G.firstVertex;
        while (P->nextVertex != NULL) {
            P = P->nextVertex;
        }
        P->nextVertex = V;
    }
}

ptrVertex searchVertex(Graph G, charInfo data) {
    ptrVertex P = G.firstVertex;
    while (P != NULL) {
        if (P->id == data) return P;
        P = P->nextVertex;
    }
    return NULL;
}

void addEdge(ptrVertex v1, ptrVertex v2) {
    if (v1 == NULL || v2 == NULL) return;

    ptrEdge e1 = alokasiEdge(v2);
    e1->nextEdge = v1->firstIncidentEdge;
    v1->firstIncidentEdge = e1;

    ptrEdge e2 = alokasiEdge(v1);
    e2->nextEdge = v2->firstIncidentEdge;
    v2->firstIncidentEdge = e2;
}

void showGraphData(Graph G) {
    ptrVertex v = G.firstVertex;
    while (v != NULL) {
        cout << "[" << v->id << "]" terhubung ke -> ";

        ptrEdge e = v->firstIncidentEdge;
        if (e == NULL) cout << "(tidak ada)";

        while (e != NULL) {
            cout << e->destVertex->id << " ";
            e = e->nextEdge;
        }
        cout << endl;
        v = v->nextVertex;
    }
}

void resetStatus(Graph G) {
    ptrVertex p = G.firstVertex;

```

```

    while (p != NULL) {
        p->status = 0;
        p = p->nextVertex;
    }
}

void runDFS(ptrVertex V) {
    if (V->status == 1) return;

    V->status = 1;
    cout << V->id << " ";

    ptrEdge e = V->firstIncidentEdge;
    while (e != NULL) {
        if (e->destVertex->status == 0) {
            runDFS(e->destVertex);
        }
        e = e->nextEdge;
    }
}

void executeDFS(Graph G, ptrVertex startV) {
    if (startV == NULL) return;

    resetStatus(G);
    cout << "DFS Traversal: ";
    runDFS(startV);
    cout << endl;
}

void executeBFS(Graph G, ptrVertex startV) {
    if (startV == NULL) return;

    resetStatus(G);
    cout << "BFS Traversal: ";

    QueueList Q;
    createQ(Q);

    startV->status = 1;
    enq(Q, startV);

    while (!emptyQ(Q)) {
        ptrVertex cur = deq(Q);
        cout << cur->id << " ";

        ptrEdge e = cur->firstIncidentEdge;
        while (e != NULL) {
            if (e->destVertex->status == 0) {
                e->destVertex->status = 1;
            }
        }
    }
}

```

```

        enq(Q, e->destVertex);
    }
    e = e->nextEdge;
}
}
cout << endl;
}

int main() {
    Graph G;
    initGraph(G);

    char listV[] = {'A','B','C','D','E','F','G','H'};
    for (int i = 0; i < 8; i++) {
        addVertex(G, listV[i]);
    }

    ptrVertex A = searchVertex(G,'A');
    ptrVertex B = searchVertex(G,'B');
    ptrVertex C = searchVertex(G,'C');
    ptrVertex D = searchVertex(G,'D');
    ptrVertex E = searchVertex(G,'E');
    ptrVertex F = searchVertex(G,'F');
    ptrVertex Gv = searchVertex(G,'G');
    ptrVertex H = searchVertex(G,'H');

    addEdge(A,B); addEdge(A,C);
    addEdge(B,D); addEdge(B,E);
    addEdge(C,F); addEdge(C,Gv);
    addEdge(D,H); addEdge(E,H);
    addEdge(F,H); addEdge(Gv,H);

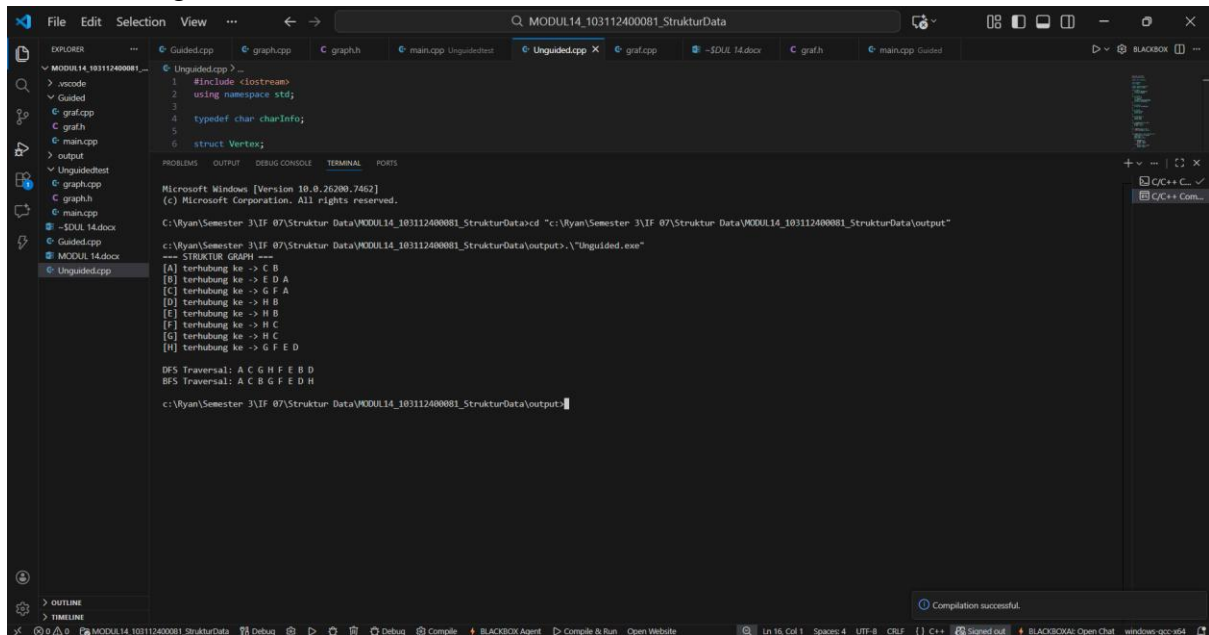
    cout << "=== STRUKTUR GRAPH ===" << endl;
    showGraphData(G);
    cout << endl;

    executeDFS(G, A);
    executeBFS(G, A);

    return 0;
}

```

Hasil Coding



```
1 #include <iostream>
2 using namespace std;
3
4 typedef char charInfo;
5
6 struct Vertex;
7
8 struct Edge {
9     charInfo source;
10    charInfo target;
11};
12
13 struct Graph {
14     int vertexCount;
15     int edgeCount;
16     Vertex* vertices;
17     Edge* edges;
18};
19
20 void addVertex(Graph* g, charInfo v) {
21     g->vertices[g->vertexCount] = Vertex(v);
22     g->vertexCount++;
23}
24
25 void addEdge(Graph* g, charInfo s, charInfo t) {
26     g->edges[g->edgeCount] = Edge(s, t);
27     g->edgeCount++;
28}
29
30 void DFS(Graph* g, charInfo start, bool* visited) {
31     visited[start] = true;
32     cout << start << " ";
33     for (int i = 0; i < g->edgeCount; i++) {
34         if (g->edges[i].source == start && !visited[g->edges[i].target]) {
35             DFS(g, g->edges[i].target, visited);
36         }
37     }
38}
39
40 void BFS(Graph* g, charInfo start) {
41     bool* visited = new bool[g->vertexCount];
42     queue<charInfo> q;
43     q.push(start);
44     visited[start] = true;
45     while (!q.empty()) {
46         charInfo v = q.front();
47         q.pop();
48         cout << v << " ";
49         for (int i = 0; i < g->edgeCount; i++) {
50             if (g->edges[i].source == v && !visited[g->edges[i].target]) {
51                 q.push(g->edges[i].target);
52                 visited[g->edges[i].target] = true;
53             }
54         }
55     }
56}
57
58 int main() {
59     Graph g;
60     addVertex(&g, 'A');
61     addVertex(&g, 'B');
62     addVertex(&g, 'C');
63     addVertex(&g, 'D');
64     addVertex(&g, 'E');
65     addVertex(&g, 'F');
66     addVertex(&g, 'H');
67     addEdge(&g, 'A', 'B');
68     addEdge(&g, 'A', 'C');
69     addEdge(&g, 'B', 'D');
70     addEdge(&g, 'C', 'D');
71     addEdge(&g, 'D', 'E');
72     addEdge(&g, 'E', 'F');
73     addEdge(&g, 'F', 'H');
74     DFS(&g, 'A', new bool[g->vertexCount]);
75     BFS(&g, 'A');
76     return 0;
77}
```

Microsoft Windows [Version 10.0.26280.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Ryan\Semester 3\IF 07\Struktur Data\MODUL14_103112400081_StrukturData> cd "c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL14_103112400081_StrukturData\output"

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL14_103112400081_StrukturData\output> .\Unguided.exe

--- STRUKTUR GRAPH ---
[A] terhubung ke -> C B A
[B] terhubung ke -> E D A
[C] terhubung ke -> G F A
[D] terhubung ke -> H B
[E] terhubung ke -> H B
[F] terhubung ke -> H C
[G] terhubung ke -> H C
[H] terhubung ke -> G F E D
DFS Traversal: A C G H F E B D
BFS Traversal: A C B G F E D H
c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL14_103112400081_StrukturData\output>

Compilation successful.

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita membuat dan mengimplementasikan struktur data Graph menggunakan bahasa pemrograman C++ dengan metode Adjacency List. Program ini dirancang untuk:

- 1. Merepresentasikan sebuah graph tidak berarah (undirected graph)**
Setiap simpul (vertex) dapat terhubung dengan simpul lain melalui sisi (edge) dua arah.
- 2. Mengelola data graph secara dinamis**
Program memungkinkan:
 - Menambahkan vertex ke dalam graph
 - Menghubungkan dua vertex menggunakan edge
 - Menampilkan hubungan antar vertex
- 3. Menerapkan algoritma penelusuran graph**
Program mengimplementasikan dua metode traversal, yaitu:
 - Depth First Search (DFS)** untuk menelusuri graph secara mendalam
 - Breadth First Search (BFS)** untuk menelusuri graph secara melebar menggunakan struktur queue
- 4. Memahami konsep traversal dan status kunjungan vertex**
Setiap vertex memiliki status yang menandakan apakah sudah dikunjungi atau belum, sehingga tidak terjadi pengulangan saat traversal.

5. **Sebagai latihan penerapan struktur data lanjutan**

Program ini bertujuan membantu mahasiswa memahami konsep graph, pointer, linked list, serta implementasi algoritma DFS dan BFS dalam C++.

Singkatnya sih, Tujuan program ini adalah untuk mengimplementasikan struktur data graph tidak berarah menggunakan bahasa C++ serta menerapkan algoritma DFS dan BFS untuk menelusuri hubungan antar vertex dalam graph.